

Activity 2: Critical Inquiry

1. “Infrastructure is fluid”

According to me , what the author want to convey by saying that infrastructure is fluid is that it is ephemeral and always changing.

In practice it means that it can be autoscaled, has features like dynamic networking and pods which are ephemeral.

2. “Failure is constant”

By saying that failure is constant the author tries to convey that no matter how good the infrastructure and code might be, at scale there will always be situations where the system will fail. It happens mainly due to hardware , network or even cloud provider glitches.

Therefore a system must be stateless so that it auto heals itself upon failure.

3. Microservice Vs Appliance

According to me from Figure 1 :

M1–M6 (Data Monitors) and A1–A5 (Analysis Services) are true microservices because each has single responsibility , they are independently deployable and stateless which are the key characteristics of a true microservice.

AWS Simple Queue Service (SQS) and D1–D3 (DynamoDB, CosmosDB, are appliances because they are stateful therefore not a true microservice rather an appliance.

4. Challenges

Observability in a cloud-native microservices architecture introduces several non-trivial challenges across pillars of logs, metrics, and tracing.

Logs : In a system where services run as short-lived containers or pods, log entries often vanish the moment an instance is terminated. Without a consistent request or trace identifier propagated through every service, correlating a sequence of log messages into a single end-to-end transaction becomes hard. Apart from that, when different teams write logs in different format like Json,TXT, centralizing, parsing, and searching those logs at scale requires careful standardization and powerful pipelines.

Metrics : Metrics collectors typically scrape each instance at regular intervals, but if a pod terminates between scrapes, its performance data is lost unless a buffering mechanism is in place. Even with buffering, tagging metrics with dynamic labels such as pod name, node identifier can produce extremely high cardinality. That explosion in unique time-series not only degrades query performance but also drives up storage and processing costs in metric backends.

Tracing : Distributed traces rely on propagating a common context through every hop of a request across dozens of microservices and external components like databases. Any service that fails to forward that context leaves gaps in the trace, making it impossible to reconstruct the full request path.